COMP202
Complexity of Algorithms

Dynamic programming, memoization, greedy algorithms

Reading materials: Exercises 4.4 and 16.2-2, Sections 16.1, 16.2 in CLRS

Learning Outcomes

At the conclusion of this set of lecture notes, you should:

1. Understand the general ideas behind the method of dynamic programming and memoization, see their importance and be able to recognise when it can be applied.
2. Understand the use of generating functions.
3. Understand the idea of greedy algorithms, and also realize that they are not always appropriate.

Fundamental Techniques

We want to examine some general algorithmic tools that can be applied to a wide variety of different problems.

These tools include

- the Greedy Method,
- Divide and Conquer (seen already in some of the sorting algorithms), and
- Dynamic Programming.

Dynamic Programming

The *Dynamic Programming* technique is somewhat similar to Divide-and-Conquer algorithms.

The main difference is that (possibly) repetitive recursive calls are replaced by a reference to already computed values stored in a *special table*.

Dynamic Programming (cont.)

Dynamic programming is primarily used for optimization problems (maximizing or minimizing some function).

It is often applied where a *brute force* search for optimal value is *infeasible*.

However, dynamic programming is efficient only if the problem has a certain amount of structure that can be exploited.

Dynamic Programming (cont.)

An effective dynamic programming solution depends on the following factors:

- *Simple sub-problems*: There must be a way of breaking the whole problem into smaller sub-problems which have a similar structure.
- *Sub-problem optimality*: An optimal solution to the global problem must be composed of optimal solutions to a sub-problem, or collection of sub-problems.
- *Sub-problem overlap*: Optimal solutions to some sub-problems can themselves contain sub-problems in common (and often do).

Dynamic Programming (cont.)

A typical approach to solve a problem using Dynamic Programming is to first find a *recursive* solution to the problem.

Once this recursive solution is found, we then try to determine a way to compute solutions to the subproblems “from the bottom up”, so that we *avoid recomputing solutions* many times.

$\{0, 1\}$ Knapsack Problem

Let S be a set of n items, where each item i has a *positive benefit* b_i and a positive *weight* w_i.

Goal: Find the *maximum benefit* subset that does not exceed total weight W.

We have the following objective:

Find a subset $T \subseteq S$ that

$$\text{maximizes} \sum_{i \in T} b_i$$

$$\text{subject to} \sum_{i \in T} w_i \leq W_{\text{max}},$$

where W_{max} is the maximum total allowed weight in the knapsack.

$\{0, 1\}$ Knapsack Problem

Exponential solution: We can "easily" solve the $\{0, 1\}$ knapsack problem in $O(2^n)$ time by *testing all possible* subsets of the n items.

Unfortunately, *exponential complexity* is unacceptable for large n, so we then want to focus on nice characterizations for sub-problems in order to use dynamic programming.

$\{0, 1\}$ Knapsack problem (cont.)

We said before that we're looking for a recursive solution, so consider an optimal set of items $I \subseteq S$, ignoring for the moment that we don't know what exactly is in I.

The obvious (but important) thing to notice is that either I contains item n, or it doesn't.

If I *does not* use item n, then the optimal solution for S is the same as the optimal solution for the set $\{1, 2, \ldots, n-1\}$.

If I does use item n, then the optimal solution consists of item n, together with an optimal solution for the set $\{1, 2, \ldots, n-1\}$, but with a new maximum allowed weight of $W_{\max} - w_n$.

$\{0, 1\}$ Knapsack problem (cont.)

To more precisely define this recursive solution, let $S_k = \{1, 2, \ldots, k\}$ denote the set containing the first k items, and define $S_0 = \emptyset$.

Let $B[k, w]$ be the *maximum total benefit* obtained using a subset of S_k, and having total weight *at most* w.

Then we define $B[0, w] = 0$, for each $w \leq W_{\max}$, and

$$B[k, w] = \begin{cases} B[k-1, w] & \text{if } w < w_k \\ \max\{B[k-1, w],\ b_k + B[k-1, w - w_k]\} & \text{otherwise.} \end{cases}$$

Our desired solution is then $B[n, W_{\max}]$.

$\{0, 1\}$ Knapsack problem (cont.)

To make our solution as easy as possible to find, we will compute these values "from the bottom up", i.e. first we compute $B[1, w]$ for each $w \leq W_{\max}$.

Then we use those values to find $B[2, w]$ for all values of w, and so forth.

$\{0, 1\}$ Knapsack Problem - Algorithm

```
01 KNAPSACK(S, W)
     ▷ Input: Set S of n items with weights w_i, benefits b_i,
       and a total weight W.
     ▷ Output: A subset T, of S with weight at most W.
 1   for w ← 0 to W
 2         do
 3            B[0, w] ← 0
 4   for k ← 1 to n
 5         do
 6            for w ← 0 to w_k − 1 do B[k, w] ← B[k − 1, w]
 7            for w ← W downto w_k
 8                 do
 9                    if B[k − 1, w − w_k] + b_k > B[k − 1, w] then
10                          B[k, w] ← B[k − 1, w − w_k] + b_k
11                    else
12                          B[k, w] ← B[k − 1, w]
```

$\{0, 1\}$ Knapsack problem

The running time of 01KNAPSACK is dominated by the two nested "for" loops, where the outer loop iterates n times, and the inner loop W times.

Theorem: The 01KNAPSACK algorithm finds a *highest benefit subset* of S with total weight at most W in $O(nW)$ time.

[Note that the algorithm can be easily modified to also return the *set* of items that gives the maximum benefit. How?]

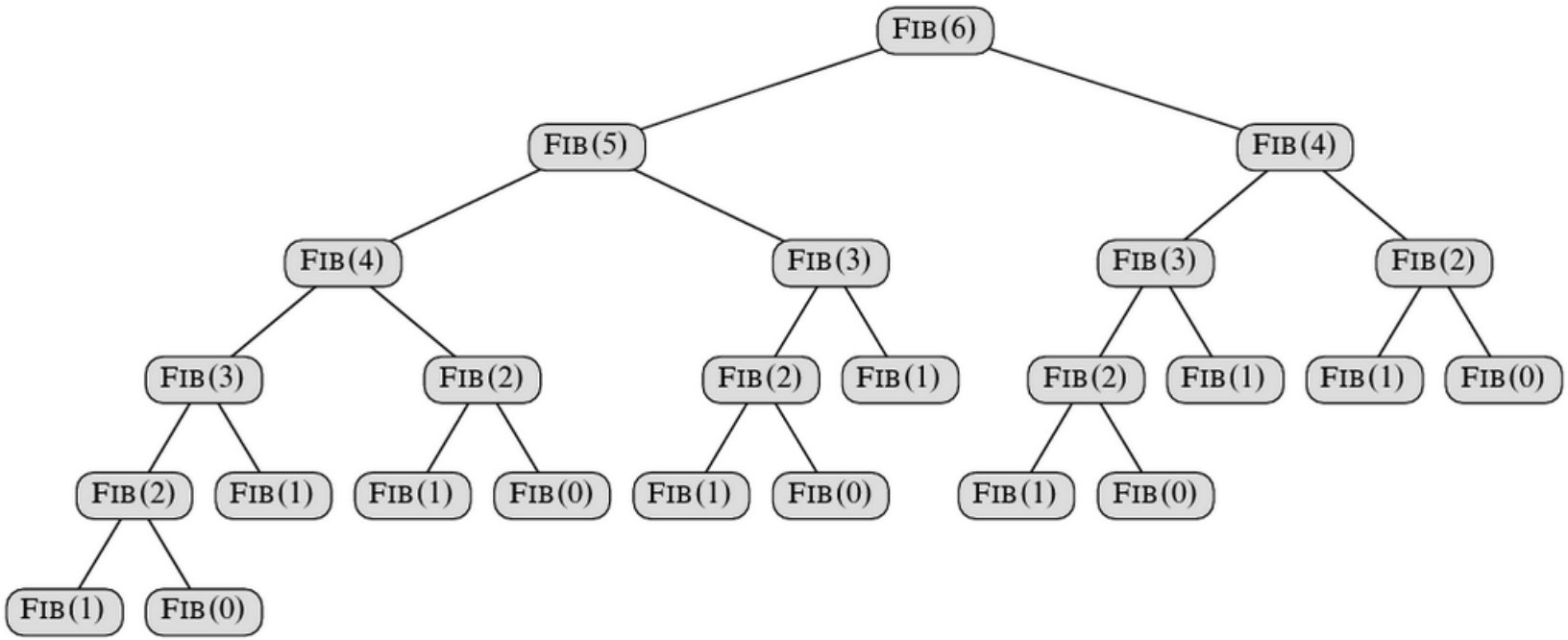
Fibonacci numbers

Define $F(1) = 1, F(2) = 1$ and $F(n) = F(n-1) + F(n-2)$ for all integers $n \geq 3$.

The Fibonacci sequence looks like $1, 1, 2, 3, 5, 8, 13, 21, 34, \ldots$.

Naive algorithm

On input n, return $F(n-1)+F(n-2)$.



Memoization

The naive algorithm has running time $T(n) = T(n-1) + T(n-2)$, which, as we'll see later, grows exponentially in n (i.e., $T(n) = c^n$ for some constant $c > 1$).

We're recomputing too much!

A possible solution (reuse things that we've computed): Define $A = [1, 1, 0, 0, \ldots, 0]$ of length n.

For i increasing from 3 to n, set $A[i]$ (starting the indexing at 1) to equal $A[i-1] + A[i-2]$, both of which we've already computed! This gives a linear-time algorithm*.
Can we do better?

An efficient algorithm via matrix multiplication

First note that

$$\begin{pmatrix} F(i+2) \\ F(i+1) \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} F(i+1) \\ F(i) \end{pmatrix}$$

This means

$$\begin{pmatrix} F(i+3) \\ F(i+2) \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^2 \begin{pmatrix} F(i+1) \\ F(i) \end{pmatrix}$$

Continuing, you can observe (or show by induction) that

$$\begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^n = \begin{pmatrix} F(n+1) & F(n) \\ F(n) & F(n-1) \end{pmatrix}$$

Repeated squaring

Thus, assuming n is a power of 2,

$$\begin{aligned} M^1 &= M, \\ M^2 &= M \cdot M, \\ M^4 &= M^2 \cdot M^2, \\ &\vdots \\ M^n &= M^{n/2} \cdot M^{n/2}. \end{aligned}$$

There are $1 + \log n$ equations above. This gives a $O(\log n)$ time algorithm*!

A closed-form solution

One way to come up with a closed-form solution is to write

$$\begin{pmatrix} F(n+1) & F(n) \\ F(n) & F(n-1) \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^n = (PDP^{-1})^n = P \cdot D^n \cdot P^{-1}.$$

If D was diagonal, computing this would be easy (assuming we could find such P, D efficiently). This can be done using **diagonalisation**, but we won't cover this.

Generating functions

Define a **generating function**:

$$\mathcal{F}(z) = \sum_{i=0}^{\infty} F_i z^i,$$

where F_i is the i'th Fibonacci number. Note that $\mathcal{F}(z) = z + z\mathcal{F}(z) + z^2\mathcal{F}(z)$, and hence

$$\mathcal{F}(z) = \frac{z}{1 - z - z^2} = \frac{z}{(1 - \phi z)(1 - \hat{\phi} z)} = \frac{1}{\sqrt{5}} \left(\frac{1}{1 - \phi z} - \frac{1}{1 - \hat{\phi} z} \right),$$

where $\phi = \frac{1+\sqrt{5}}{2}$ and $\hat{\phi} = \frac{1-\sqrt{5}}{2}$. Using $\frac{1}{1-x} = \sum_{i=0}^{\infty} x^i$, get

$$\mathcal{F}(z) = \sum_{i=0}^{\infty} \frac{1}{\sqrt{5}} (\phi^i - \hat{\phi}^i) z^i,$$

and so $F_i = \frac{1}{\sqrt{5}} (\phi^i - \hat{\phi}^i)$.

Greedy Method

An *optimization problem* is a problem that involves searching for a configuration that *minimizes* or *maximizes* some *objective function*.

The *greedy method* solves (or tries to solve) a given optimization problem by going through a sequence of (feasible) choices.

The sequence starts from a well-understood starting configuration, and then iteratively makes the decision that *seems best* from all those that are currently possible.

Problems that have a greedy solution are said to possess the *greedy-choice property*.

Greed Works!

There are many problems that can be solved using a greedy method, such as

- Fractional Knapsack Problem
- Interval Scheduling
- Task Scheduling
- Minimum Spanning Trees (e.g. Kruskal's Algorithm, Prim's Algorithm)
- Shortest Paths (e.g. Dijkstra's Algorithm, Bellman-Ford Algorithm)
- Change Making (for *some* (but not all) sets of coins)
- Maximum Spacing k-clustering
- ...

Greed Works (well, sometimes)

- A word of warning! The greedy approach does *not always* lead to an optimal solution. (And we will see some problems for which it doesn't work.)

The greedy method is also sometimes used for some *hard* (i.e. difficult to solve) problems in order to generate *approximate solutions*.

Fractional Knapsack Problem (FKP)

Let S be a set of n items, where each item i has a *positive benefit* b_i and a positive *weight* w_i.

Goal: Find the *maximum benefit* subset that does not exceed total weight W.

In a FKP we are allowed to take *arbitrary fractions*, x_i, of each item, i.e. the solution is a set of values x_i such that

$$0 \leq x_i \leq w_i \text{ for all } i, \text{ and}$$

$$\sum_{i \in S} x_i \leq W.$$

The *total benefit* of the items taken is determined by the sum

$$\sum_{i \in S} b_i(x_i/w_i).$$

Fractional Knapsack Problem (cont.)

The general method for the FKP is to compute the *value index* for each item i, where this is defined as

$$v_i = \frac{b_i}{w_i}.$$

Then we select items to include in the knapsack, starting with the *highest value index*.

Using a heap (storing the *maximum* at the root), we can compute the value indices and build the heap in $O(n \log n)$ time.

Then, each greedy choice (which removes an item with the greatest remaining value index) requires $O(\log n)$ time.

FRACTIONALKNAPSACK(S, W)

▷ Input: Set S of items, item i has weight w_i and benefit b_i, maximum total weight W.

▷ Output: Amount x_i of each item to maximize the total benefit.

```
1   for i ← 1 to |S| do
2           x_i ← 0
3           v_i ← b_i / w_i
4           Insert (v_i, i) into a heap H (max value index at root).
5   w ← 0
6   while w < W do
7           Remove the max value from H.
8           a ← min{w_i, W − w}
9           x_i ← a
10          w ← w + a
```

Fractional Knapsack Problem (cont.)

FKP satisfies the *greedy-choice property* (why?), hence

Theorem: Given an instance of FKP with set S of n items, we can construct a maximum benefit subset of S (allowing for fractional amounts of items) in $O(n \log n)$ time.

Fractional Knapsack Problem (cont.)

FKP satisfies the *greedy-choice property* (why?)

Argument: Suppose we are left with space $r = W - w$ in the knapsack. We have two items to be packed there, i and j, where

$$\frac{b_i}{w_i} < \frac{b_j}{w_j} \text{ and } r < w_i, w_j.$$

Then choosing i contributes to the benefit of the solution $\frac{r}{w_i} \cdot b_i$, and choosing j contributes $\frac{r}{w_j} \cdot b_j$, then clearly

$$\frac{r}{w_i} \cdot b_i < \frac{r}{w_j} \cdot b_j,$$

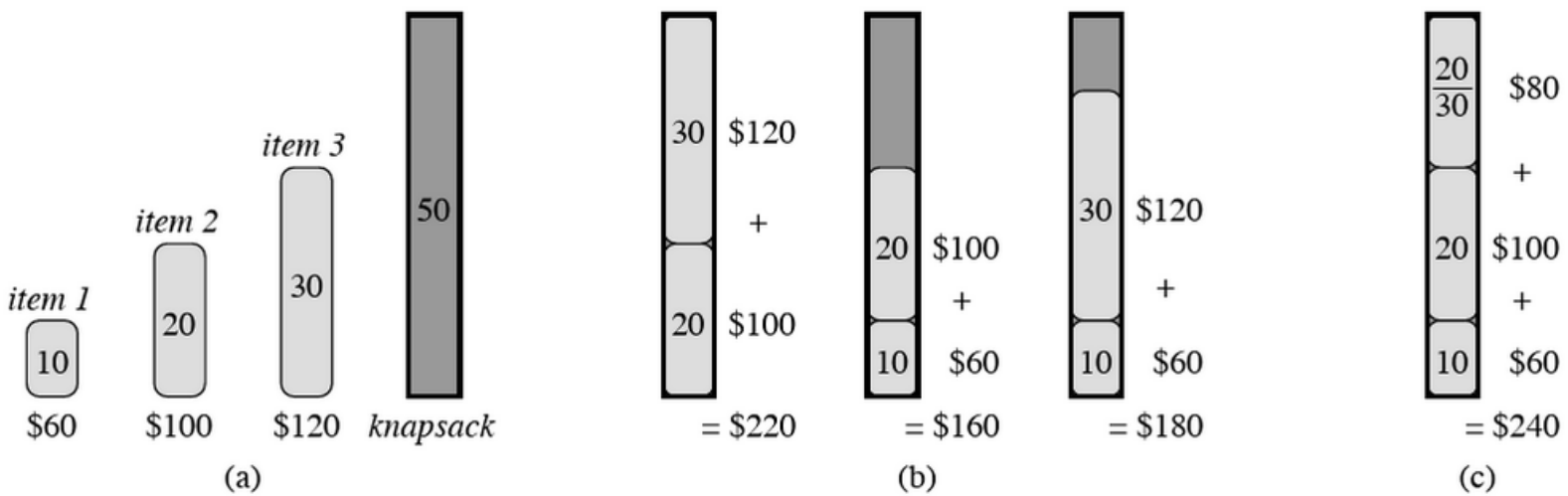
therefore we should choose j first!

Fractional vs. $\{0, 1\}$-Knapsack Problem

The $\{0, 1\}$-Knapsack Problem is very similar to the Fractional Knapsack Problem.

The significant difference is that in the $\{0, 1\}$ version, we are not allowed to take fractional amounts of the items, either we take the entire item, or leave it behind.

A Greedy Method *does not* (in general) find an optimal solution of the $\{0, 1\}$-Knapsack Problem.



Interval Scheduling

We now consider a problem sometimes referred to as the *Interval Scheduling Problem*.

Here we have a collection of tasks (or intervals of requested time to use a room, etc.), and a single machine that can process these tasks (or a single room in which meetings can be held, etc.).

Goal: We want to select a subset of the tasks in order to maximize the *number* of tasks that we can schedule on the machine.

Interval Scheduling (cont.)

More formally (and keeping with the task/machine terminology), suppose we are given a set T of n tasks, where each task i has a start time s_i and a finish (or completion) time f_i.

Two tasks i and j are *non-conflicting* (or *compatible*) if

$$f_i \leq s_j \quad \text{or} \quad f_j \leq s_i.$$

So two tasks can be executed on the machine only if they are *non-conflicting*.

Goal: Select a *maximum size* subset of non-conflicting tasks.

Interval scheduling (cont.)

Can we find a greedy algorithm for finding this solution?

There are several methods we might happen to try in our search for a method that works.

Interval Scheduling (cont.)

Suppose, for example, we always pick the first available task with the *earliest start time*.

This idea doesn't work, so we might try to *accept "short" intervals first*, i.e. take ones for which $f_i - s_i$ is smallest first.

Or, maybe we should first *accept tasks that don't "collide" with many other tasks*.

Hmmm, we seem to be running out of ideas...

Interval Scheduling (cont.)

How about accepting the task that finishes first? In other words, we want to "free up" the machine as soon as possible to start another task.

This idea works!

Order the tasks in terms of their *completion times*. Then select the task that finishes first, remove all tasks that conflict with this one, and repeat until we're done.

```
INTERVALSCHEDULE(T)
    ▷ Input: A set, T, of tasks with start times and end times.
    ▷ Output: A maximum-size subset, A, of non-conflicting tasks.
1   A ← ∅
2   while T ≠ ∅
3         do
4               Remove the task t with earliest completion time.
5               Add t to A
6               Delete all tasks from T that conflict with t
7   Return the set A as the set of scheduled tasks
```

Interval Scheduling (cont.)

Theorem: The algorithm *IntervalSchedule* returns a set of non-compatible tasks having maximum size.

Furthermore, we can make this algorithm run in time $O(n \log n)$ (the main time will lie in sorting the tasks by their completion times).

Correctness

Theorem 16.1

Consider any nonempty subproblem S_k, and let a_m be an activity in S_k with the earliest finish time. Then a_m is included in some maximum-size subset of mutually compatible activities of S_k.

Proof Let A_k be a maximum-size subset of mutually compatible activities in S_k, and let a_j be the activity in A_k with the earliest finish time. If $a_j = a_m$, we are done, since we have shown that a_m is in some maximum-size subset of mutually compatible activities of S_k. If $a_j \neq a_m$, let the set $A'_k = A_k - \{a_j\} \cup \{a_m\}$ be A_k but substituting a_m for a_j. The activities in A'_k are disjoint, which follows because the activities in A_k are disjoint, a_j is the first activity in A_k to finish, and $f_m \leq f_j$. Since $|A'_k| = |A_k|$, we conclude that A'_k is a maximum-size subset of mutually compatible activities of S_k, and it includes a_m. ■